As the code above shows, by overriding append_features you can prevent a module from being included at all. Interestingly, that's exactly what Concern wants to do, as we'll see soon.

Concern#append_features

Concern defines its own version of append_features.

gems/activesupport-4.1.0/lib/active_support/concern.rb

```
module ActiveSupport
  module Concern
    def append_features(base)
        # ...
```

Remember the *Class Extension (130)* spell? append_features is an instance method on Concern, so it becomes a class method on modules that extend Concern. For example, if a module named Validations extends Concern, then it gains a Validation.append_features class method. If this sounds confusing, look at this picture showing the relationships between Module, Concern, Validations, and Validation's singleton class:

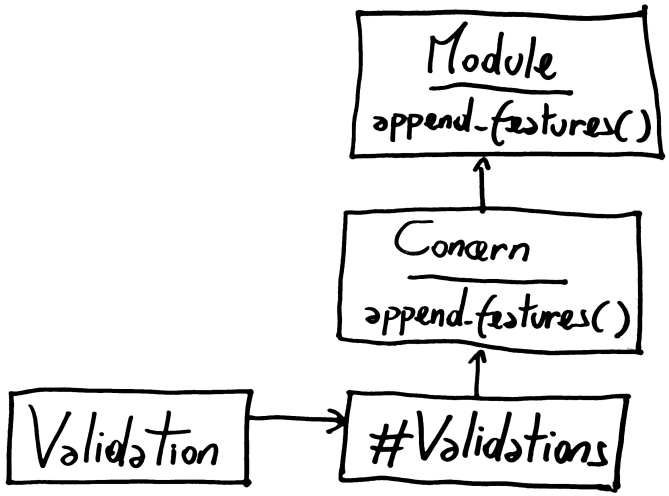


Figure 10—ActiveSupport::Concern overrides Module#append_features.

Let's recap what we've learned so far. First, modules that extend Concern get an @_dependencies Class Variable. Second, they get an override of append_features. With those two concepts in place, we can look at the code that makes Concern tick.